

Каф. ПМИ, ИМИКН

22 сентября 2026 г.

Распределенные вычисления

Лекция 3. Интерконнекты. Разработка и оценка производительности параллельных программ

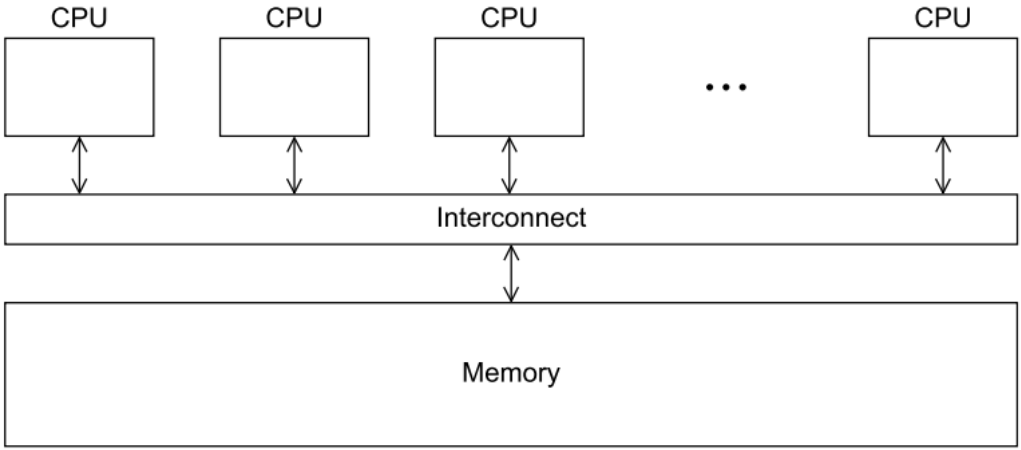
Ф.С. Пеплин

fpeplin@hse.ru

https://www.hse.ru/org/persons/749540639/



Интерконнекты — Общая память



Интерконнект — Общая память
UMA, NUMA

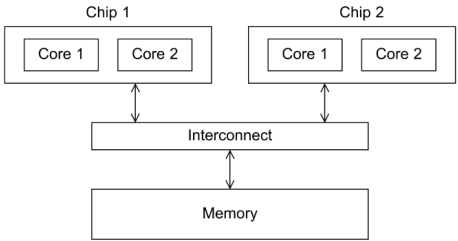


Рис.: Uniform Memory Access (UMA)

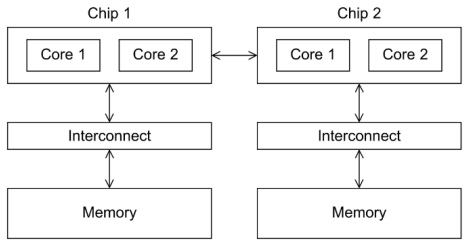


Рис.: Nonuniform Memory Access (NUMA)



Интерконнекты — Общая память

Шина

Шина

Базовый тип интерконнекта, в котором все компоненты системы используют одну общую линию передачи данных. В один момент времени передавать информацию может только одно устройство, а остальные «слушают».

Шина

Базовый тип интерконнекта, в котором все компоненты системы используют одну общую линию передачи данных. В один момент времени передавать информацию может только одно устройство, а остальные «слушают».

Ключевые особенности

- **Разделяемая среда**. Общая магистраль для процессора, памяти и периферии. Если шина занята, другие устройства ждут (требуется арбитраж трафика).
- **Широковещательность**. Данные, отправленные в шину, видны всем подключенным к ней узлам, но обрабатывает их только адресат.

Шина

Базовый тип интерконнекта, в котором все компоненты системы используют одну общую линию передачи данных. В один момент времени передавать информацию может только одно устройство, а остальные «слушают».

Ключевые особенности

- **Разделяемая среда**. Общая магистраль для процессора, памяти и периферии. Если шина занята, другие устройства ждут (требуется арбитраж трафика).
- **Широковещательность**. Данные, отправленные в шину, видны всем подключенным к ней узлам, но обрабатывает их только адресат.

Преимущества

- Простота и дешевизна
- Гибкость

Шина

Базовый тип интерконнекта, в котором все компоненты системы используют одну общую линию передачи данных. В один момент времени передавать информацию может только одно устройство, а остальные «слушают».

Ключевые особенности

- **Разделяемая среда**. Общая магистраль для процессора, памяти и периферии. Если шина занята, другие устройства ждут (требуется арбитраж трафика).
- **Широковещательность**. Данные, отправленные в шину, видны всем подключенным к ней узлам, но обрабатывает их только адресат.

Преимущества

- Простота и дешевизна
- Гибкость

Недостатки

- Ограниченная масштабируемость
- Низкая отказоустойчивость



Интерконнекты — Общая память

Crossbar

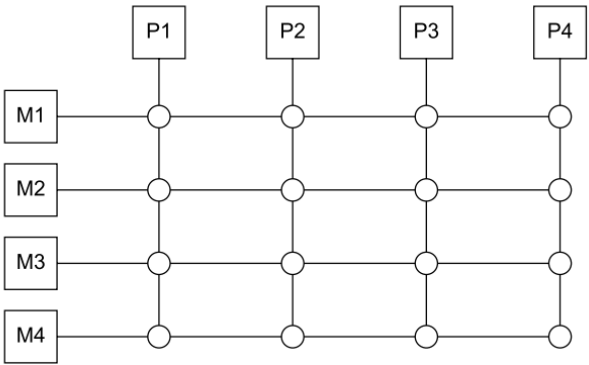


Рис.: Кроссбар

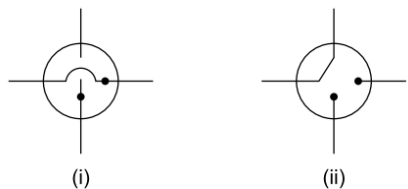
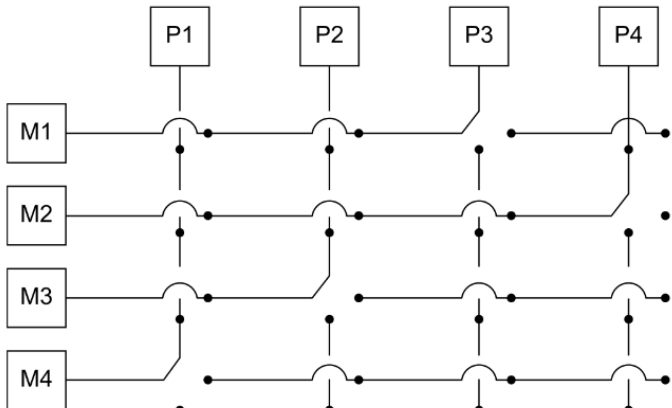


Рис.: Кросспоинт

Интерконнекты — Общая память

Crossbar



Преимущества

- **Максимальная пропускная способность**. Кроссбар является неблокируемой структурой. Это значит, что любые пары «вход – выход» могут передавать данные одновременно, если выходы не совпадают.

Преимущества

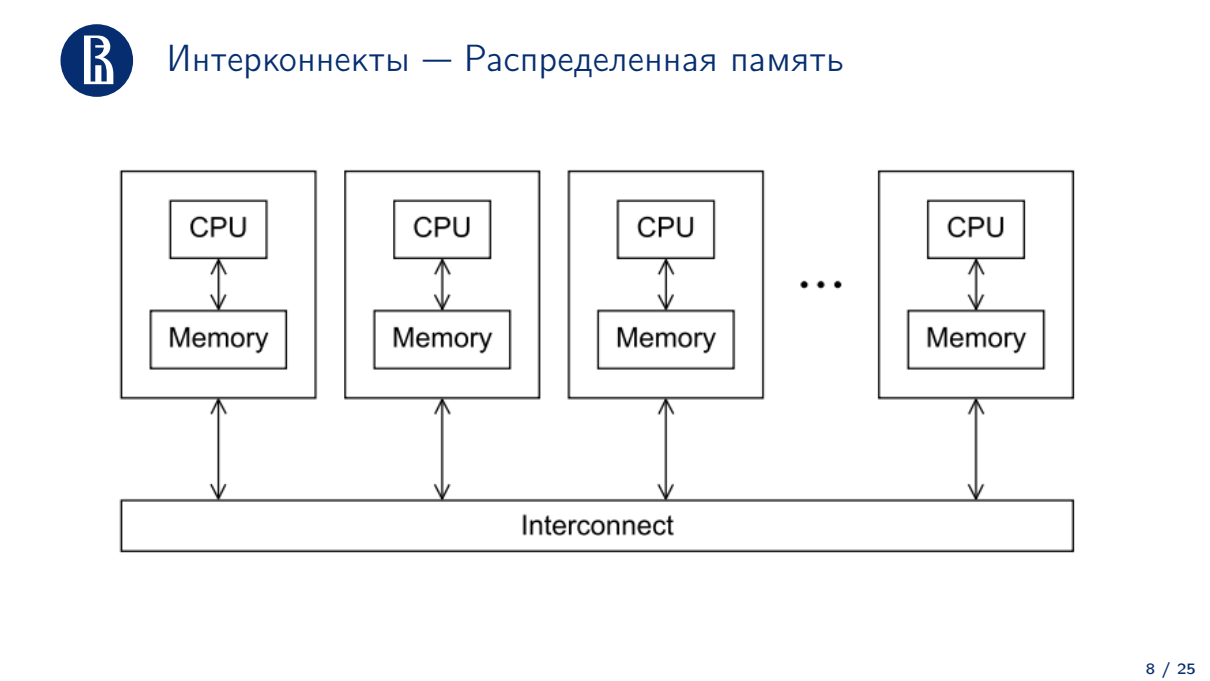
- **Максимальная пропускная способность**. Кроссбар является неблокируемой структурой. Это значит, что любые пары «вход – выход» могут передавать данные одновременно, если выходы не совпадают.
- **Минимальная задержка**. Данные проходят напрямую от входа к выходу всего через один коммутационный узел, что обеспечивает фиксированное и очень низкое время задержки. Отсутствие конкуренции за промежуточные каналы связи делает производительность стабильной и предсказуемой.

Преимущества

- **Максимальная пропускная способность**. Кроссбар является неблокируемой структурой. Это значит, что любые пары «вход – выход» могут передавать данные одновременно, если выходы не совпадают.
- **Минимальная задержка**. Данные проходят напрямую от входа к выходу всего через один коммутационный узел, что обеспечивает фиксированное и очень низкое время задержки. Отсутствие конкуренции за промежуточные каналы связи делает производительность стабильной и предсказуемой.

Недостатки

Высокие стоимость и энергопотребление



Интерконнекты — Распределенная память

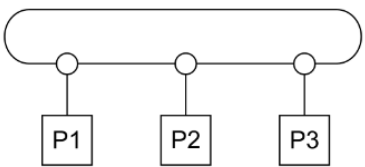


Рис.: Кольцо

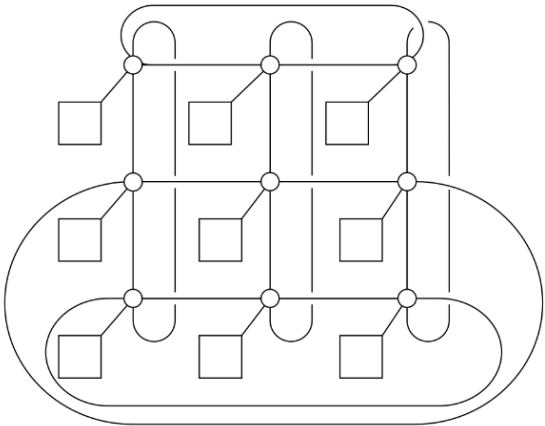


Рис.: Тороидальная сетка

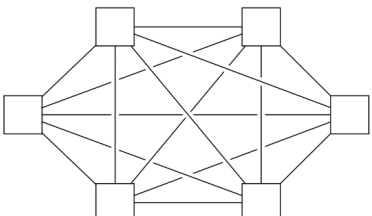


Рис.: Полностью связанный интерконнект

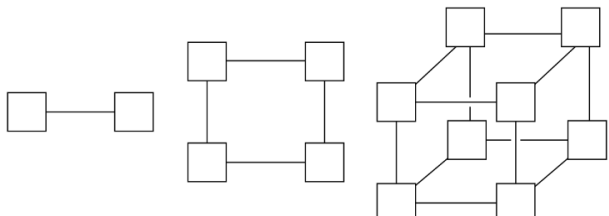


Рис.: Гиперкуб

Ширина бисекции — два эквивалетных определения

- Разбить систему на два множества с равным количеством узлов. Определить количество потоков данных, которые могут одновременно пересекать границу.
- Определить минимальное количество связей, которое нужно удалить, чтобы разбить систему на два несвязанных множества с одинаковым количеством узлов.



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо			



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2		



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка			



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$		



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть			



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$		



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p
Гиперкуб			



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p
Гиперкуб	$\frac{p}{2}$		



Интерконнекты — распределенная память

Сравнение интерконнектов

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p
Гиперкуб	$\frac{p}{2}$	$\frac{1}{2} p \log_2 p$	

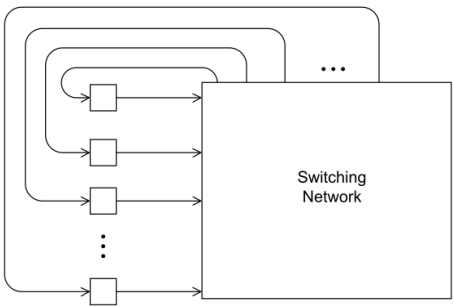


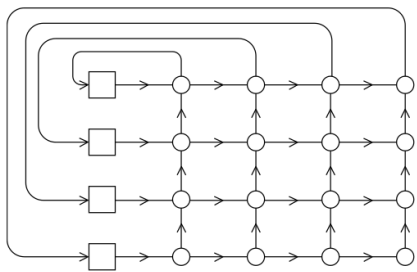
Интерконнекты — распределенная память

Сравнение интерконнектов

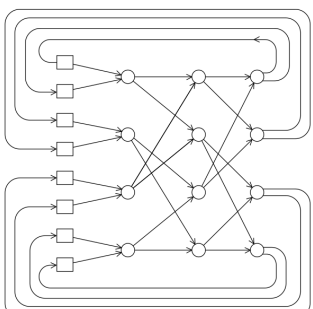
Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p
Гиперкуб	$\frac{p}{2}$	$\frac{1}{2} p \log_2 p$	$1 + \log_2 p$

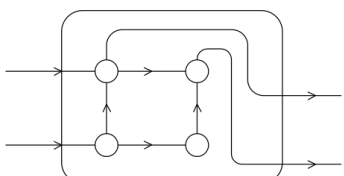
Кроссбар





Омега







Разработка параллельных программ

Вызовы, с которыми сталкивается ~~вайбкодер~~ программист при разработке параллельных программ

1. Состояние гонки
2. False sharing
3. Недетерминизм
4. Потокобезопасность
5. Сложность отладки
6. Балансировка нагрузки
7. Высокие накладные расходы на синхронизацию

Приведенные вызовы, кроме первых трех, актуальны также для программирования систем с распределенной памятью.



Состояние гонки

sum находится в общей памяти и значение равно 100000.

1	2
sum += 1000000	sum -= 30000

t	1	2
1	Получить	—
2	Сравнить экспоненты	Получить
3	Сдвинуть запятую	Сравнить экспоненты
4	Сложить мантиссы	Сдвинуть запятую
5	Округлить	Сложить мантиссы
6	Записать	Округлить
7	—	Записать



Когерентность кэшей

Пусть $x = 7$ — общая переменная для двух процессоров, a — переменная процессора 1, b, c — переменные процессора 2.

Время	Процессор 1	Процессор 2
0	$a = 2x$	$c = x$
1	$x = 12$	Команды без x
2	—	$b = 10x$

Чему равно b?



Когерентность кэшей

Пусть $x = 7$ — общая переменная для двух процессоров, a — переменная процессора 1, b, c — переменные процессора 2.

Время	Процессор 1	Процессор 2
0	$a = 2x$	$c = x$
1	$x = 12$	Команды без x
2	—	$b = 10x$

Чему равно b?

Способы обеспечения когерентности кэшей

1. Отслеживание
2. Справочник



Когерентность кэшей

Пусть $x = 7$ — общая переменная для двух процессоров, a — переменная процессора 1, b, c — переменные процессора 2.

Время	Процессор 1	Процессор 2
0	$a = 2x$	$c = x$
1	$x = 12$	Команды без x
2	—	$b = 10x$

Чему равно b?

Способы обеспечения когерентности кэшей

1. Отслеживание
2. Справочник

При обеспечении когерентности кэшей может возникнуть **false sharing**, когда процессор вынужден заново загружать кэш-строку, даже если та ее часть, с которой работает данное ядро, не менялась.

Message Passing

```
char message[100];
my_rank = Get_rank();
if(my_rank == 1){
        sprintf(message, "Hello from process 1");
        Send(message, MSG_CHAR, 100, 0);}
        else if(my_rank == 0){
                Receive(message, MSG_CHAR, 100, 1);
                printf("Process 0 > Received : %s\n" , message);
}
```



Разработка параллельных программ

Распределенная память

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.
2. Процесс 0 должен знать, когда можно безопасно записать свое значение в память процесса 1

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.
2. Процесс 0 должен знать, когда можно безопасно записать свое значение в память процесса 1
   - Нужно синхронизировать оба процесса перед записью

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.
2. Процесс 0 должен знать, когда можно безопасно записать свое значение в память процесса 1
   - Нужно синхронизировать оба процесса перед записью
3. Процесс 1 должен знать, когда можно брать значение из процесса 0

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.
2. Процесс 0 должен знать, когда можно безопасно записать свое значение в память процесса 1
   - Нужно синхронизировать оба процесса перед записью
3. Процесс 1 должен знать, когда можно брать значение из процесса 0
   - еще одна синхронизация, или

One-sided communication

Один процесс вызывает функцию, которая обновляет переменную на этом процессе значением из другого процесса, или, наоборот, записывает некоторое значение в переменную на другом процессе. Подход позволяет избавиться от накладных расходов, связанных с необходимостью связать вызовы send и receive.

Проблема

1. Пусть процесс 0 записывает значение в переменную процесса 1.
2. Процесс 0 должен знать, когда можно безопасно записать свое значение в память процесса 1
   - Нужно синхронизировать оба процесса перед записью
3. Процесс 1 должен знать, когда можно брать значение из процесса 0
   - еще одна синхронизация, или
   - держать на процессе 0 переменную-флаг, которую процесс 1 будет постоянно запрашивать

Partitioned global address space languages

Вся память системы представляется как единое глобальное адресное пространство, но при этом оно разделено на части (партиции). Каждый вычислительный узел (процессор или поток) имеет быстрый локальный доступ к своей части памяти и может напрямую обращаться к чужим (удаленным) частям без явного использования пересылки сообщений.

Осведомленность о локальности

Программист или среда исполнения понимают, какие данные лежат «близко» (в локальной памяти узла), а какие «далеко» (на другом узле через сеть).

Ускорение

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Если $S = p$, то ускорение называется линейным.

Эффективность (p — количество процессоров)

$$E = \frac{S}{p} = \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}}$$

Время работы параллельной программы

$$T_{\text{parallel}} = \frac{T_{\text{serial}}}{p} + T_{\text{overhead}}$$



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$).



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\frac{(1-r)T_{\text{serial}}}{p} + rT_{\text{serial}}}$$



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\frac{(1-r)T_{\text{serial}}}{p} + rT_{\text{serial}}} < \frac{T_{\text{serial}}}{rT_{\text{serial}}}$$



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\frac{(1-r)T_{\text{serial}}}{p} + rT_{\text{serial}}} < \frac{T_{\text{serial}}}{rT_{\text{serial}}} = \frac{1}{r}$$

Закон Амдала

Если r — доля программы, которая не может быть распараллелена, то распараллеливание может сделать программу быстрее не более, чем в $\frac{1}{r}$ раз.



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\frac{(1-r)T_{\text{serial}}}{p} + rT_{\text{serial}}} < \frac{T_{\text{serial}}}{rT_{\text{serial}}} = \frac{1}{r}$$

Закон Амдала

Если r — доля программы, которая не может быть распараллелена, то распараллеливание может сделать программу быстрее не более, чем в $\frac{1}{r}$ раз.

Замечания

1. Во многих случаях $\frac{1}{r}$ может быть достаточно большой для практических задач.



Закон Амдала (1967)

Пусть $0 < r < 1$ — часть программы, которая не поддается распараллеливанию, а весь остальной код допускает распараллеливание без накладных расходов ($T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}$). Дадим оценку ускорению S сверху.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\frac{(1-r)T_{\text{serial}}}{p} + rT_{\text{serial}}} < \frac{T_{\text{serial}}}{rT_{\text{serial}}} = \frac{1}{r}$$

Закон Амдала

Если r — доля программы, которая не может быть распараллелена, то распараллеливание может сделать программу быстрее не более, чем в $\frac{1}{r}$ раз.

Замечания

1. Во многих случаях $\frac{1}{r}$ может быть достаточно большой для практических задач.
2. При росте размера задачи r обычно уменьшается.



Закон Густавсона (1988)

Закон Густавсона-Барсиса

Ускорение масштабирования S_p

$$S_p = (1 - r)p + r$$



Закон Густавсона (1988)

Закон Густавсона-Барсиса

Ускорение масштабирования S_p

$$S_p = (1 - r)p + r$$

Содержательный смысл величины S_p



Закон Густавсона (1988)

Закон Густавсона-Барсиса

Ускорение масштабирования S_p

$$S_p = (1 - r)p + r$$

Содержательный смысл величины S_p

Отношение количества инструкций, выполненных за единицу времени на параллельной архитектуре, к количеству инструкций, выполненных за тот же промежуток времени программой без распараллеливания.

Формулировка закона Густавсона

Если r мало, то объем работы, выполненной параллельной программой, почти в p раз больше, чем объем работы, выполненный программой без распараллеливания.



Масштабирование параллельных программ

Масштабируемая программа

При увеличении количества процессов/потоков можно увеличить размер входных данных так, чтобы эффективность E не изменилась

Сильная масштабируемость

Эффективность E не изменяется при том же размере входных данных

Слабая масштабируемость

Эффективность E не изменяется, если размер входных данных изменяется также, как и количесто потоков/процессов



Одновременный, параллельный, распределенный

Одновременные вычисления (Concurrent computing)

Различные задачи могут быть запущены одновременно.

Параллельные вычисления (Parallel computing)

Различные процессы работают одновременно на разных вычислительных устройствах и взаимодействуют друг с другом для решения задачи.

Распределенные вычисления (Distributed computing)

Программы взаимодействуют с другими программами для решения задачи.